Project 03: Bayes Nets

CSE 537 - Artificial Intelligence

Probabilistic inference implementation and analysis

Param Shah: 117369485

Madhura Chitale : 117420795

1. Implementation and evaluation method

We implemented Questions 1 to 4 in inference.py and factorOperations.py. The work covers the Ghostbusters Bayes net, factor joining, marginalization, and variable elimination. Supplied names, signatures, constants, type checks, tracking wrappers, support files, and Berkeley attribution were retained.

Evaluation method

We ran python test_q1to4.py without modifying the supplied script. Additional checks in test_additional.py used random seed 537 and ten binary networks with structure A -> C <- B and C -> D. We compared 150 query, evidence, and elimination-order combinations with enumeration to absolute tolerance 1e-12. Operation tracking was checked separately from posterior values.

2. Question 1: Bayes Net Structure

The network contains Pacman, Ghost0, Ghost1, Observation0, and Observation1. Pacman and Ghost0 point to Observation0; Pacman and Ghost1 point to Observation1. Position domains include every (x, y) cell, ignoring walls. Observation domains contain all integers from zero through width + height - 2 + MAX_NOISE, inclusive.

Result interpretation

The supplied 4-by-3 example passes: all three position variables have 12 possible positions, and both observation variables range from 0 through 12. The function constructs the empty network; CPT probabilities are assigned separately as required.

3. Question 2: Join Factors

The result takes the union of unconditioned variables. Conditioned variables are their union across inputs minus variables already unconditioned. For each result assignment, matching input probabilities are multiplied. A new factor preserves the complete domain dictionary and does not normalize the entries.

The supplied Badge/Alarm test passes. An additional check joins unnormalized factors with a shared conditioned variable: entries 2 and 3 produce entries 6, confirming that joining does not assume normalized CPTs.

4. Question 3: Eliminate

Marginalization implementation

Elimination creates a new factor whose unconditioned set excludes the selected variable. For each retained assignment, the implementation sums the input probabilities over every value in the eliminated variable domain. Conditioned variables and the complete domain dictionary remain unchanged. Supplied checks reject eliminating a conditioned variable or the only unconditioned variable.

Test results

The supplied test sums Alarm out of a factor over Badge and Alarm conditioned on Zone. It returns 0.35 and 0.65 for green and gold in the quiet zone, and 0.55 and 0.45 in the busy zone. The additional unnormalized-factor check sums two entries of 6 to obtain 12 and verifies the preserved domains.

5. Question 4: Variable Elimination

Join and eliminate sequence

The implementation first obtains CPTs restricted to the observed evidence. For each hidden variable in the requested order, it joins all factors containing that variable and immediately eliminates it. A joined factor with only one unconditioned variable is discarded as required. The remaining factors are joined and normalized using the supplied normalize function.

When no elimination order is supplied, the existing wrapper selects a sorted order of hidden variables. The implementation uses the supplied tracking functions, so each recorded elimination immediately follows the join for the same variable.

Fuel state	P(state, Lamp=lit)	P(state | Lamp=lit)
full	0.222	111/202 = 0.549505
low	0.182	91/202 = 0.450495

Table 1. Supplied Fuel -> Sensor -> Lamp query, eliminating Sensor.

Behavior and result interpretation

The supplied posterior test passes. The joint probability of a lit lamp is 0.404; dividing each joint entry by this total produces a posterior that sums to one. Evidence remains represented by its restricted domain. An independent hidden root produces only a join record and is discarded without an eliminate call. A zero-mass result returns None through the supplied normalization behavior.

6. Validation results

Check	Result
Supplied Q1-Q4 examples	4/4 PASS
Posterior and operation-order comparisons	150/150 PASS
Default orders and independent-root discard	PASS
Unnormalized factors and retained domains	PASS
Zero-mass normalization behavior	PASS

Table 2. Results from test_q1to4.py and test_additional.py.

7. Critical analysis of variable elimination

Correctness and computational work

All 150 posterior comparisons agree with enumeration to absolute tolerance 1e-12 on the tested models. Enumeration joins the full network before marginalizing hidden variables. Variable elimination interleaves these operations, allowing variables to be summed out before unrelated factors are joined. Both compute the same conditional distribution on these examples, while their intermediate tables can differ substantially.

Effect of elimination order and evidence

Consider the binary network A -> C <- B and C -> D, querying A with D observed. Eliminating B first joins scopes {B} and {A,B,C}, then sums out B. The next join has scope {A,C,D}. Eliminating C first joins {A,B,C} and {C,D}, creating scope {A,B,C,D} before marginalization.

Order	Largest joined scope	Rows before / after evidence
B, C	{A,B,C} or {A,C,D}	8 / 8
C, B	{A,B,C,D}	16 / 8

Table 3. Analytical peak joined-table sizes for binary domains and fixed D.

Without evidence restriction, the four-variable join has 16 entries rather than 8. Restricting D to one value reduces it to 8. In this example the two orders therefore have the same peak joined-table size after evidence, even though their scopes differ. The example shows why both order and actual domain sizes must be considered when assessing work; it does not demonstrate a measured runtime improvement.

Complexity and limitations

An intermediate factor requires storage proportional to the product of its variable-domain sizes. Dense connectivity or an unfavorable order can therefore make variable elimination exponential in the largest intermediate scope. Early marginalization can save substantial work, but it does not guarantee small factors. This implementation follows the requested elimination order and does not search for an optimized order.

The randomized checks cover small binary models and several queries, evidence sets, and orders. Enumeration is a useful reference for scheduling and posterior comparisons, but it shares the implemented join and eliminate primitives. The supplied independently calculated examples and the additional constant-factor checks provide separate numerical checks for those primitives.